

1. Personal Introduction (My Intro)

Hi, I'm Md Riyad Hossain. I am a Computer Science graduate and a Junior Backend Developer. I love solving problems and building scalable software. I have practical experience working with fullstack technologies. On the backend, I mainly use Golang and C# with ASP.NET Core. For the frontend, I work with JavaScript, TypeScript, and Next.js. In my current remote job, I build microservices, manage databases like PostgreSQL, and use Docker for containerization. I also enjoy competitive programming and have solved over 350 problems on Codeforces and LeetCode.

2. Core Technical & Behavioral Q&A;

- Category: JS/TS Proficiency

Question 1: What is the difference between 'let' and 'const' in JavaScript?

Answer: Both 'let' and 'const' are block-scoped variables. We use 'let' when we want to change or reassign the value of the variable later. We use 'const' when the value should stay the same and never change. In clean code, it is best to use 'const' by default unless we really need to change the Value.

Question 2: What is an Asynchronous function in JavaScript?

Answer: An asynchronous function allows the program to start a long task and move to the next task without waiting. For example, fetching data from an API takes time. Instead of freezing the screen, JavaScript handles the API call in the background. We use 'async' and 'await' keywords to write this code cleanly.

Question 3: Why do we use TypeScript if we already know JavaScript?

Answer: We use TypeScript because it adds static typing to JavaScript. In JavaScript, we can accidentally pass a string into a function that expects a number, which causes bugs. TypeScript finds these type mistakes while we are writing the code, before the application runs. This makes our code safer and easier to maintain.

- Category: Fullstack Fundamentals

Question 4: What is the difference between client-side rendering (CSR) and server-side rendering (SSR)?

Answer: In Client-Side Rendering, the browser downloads a minimal HTML page and JavaScript builds the content on the user's screen. In Server-Side Rendering, the server builds the complete HTML page with data first and then sends it to the browser. SSR is much faster for the first page load and is much better for SEO.

Question 5: What is CORS error and how do you fix it?

Answer: CORS stands for Cross-Origin Resource Sharing. It is a security feature in web browsers.

It blocks a frontend website from accessing a backend server if they are on different links or ports. To fix this, we must configure our backend server to allow and trust the frontend link by adding proper CORS headers.

Question 6: How do you keep user sessions secure in a fullstack application?

Answer: We usually use JSON Web Tokens, also known as JWT. When a user logs in, the backend creates a secure token and sends it to the frontend. The frontend stores this token safely, usually in an HTTP-only cookie. For every next API request, the frontend sends this token so the backend can verify the user.

- Category: Problem Solving & Debugging

Question 7: If your web application is loading very slowly, how do you find the problem?

Answer: First, I open the browser Developer Tools and look at the Network tab. I check which API call or image is taking too much time. If the backend API is slow, I check the database queries to see if we need indexing. If the frontend is slow, I check if we are loading too many heavy JavaScript packages at once.

Question 8: How do you avoid writing nested loops that make your code slow?

Answer: Nested loops usually take $O(n^2)$ time complexity, which is very slow for large data. To fix this, I look for ways to use a Hash Map or a Set. By storing the data inside an object first, I can look up values in $O(1)$ time. This allows me to solve the problem using separate, single loops, which takes only $O(n)$ time.

- Category: Ownership & Learning Velocity

Question 9: How do you handle a situation where you disagree with a teammate about a technical choice?

Answer: When I disagree, I do not make it personal. I listen to their ideas first to understand their point of view. Then, I share my thoughts based on facts, like code performance, security, or development time. If we still cannot agree, we can build a quick prototype of both ideas or ask our team leader to help us make the best decision for the project.

Question 10: As a new graduate, how do you make sure you learn new things quickly on the job?

Answer: I am highly self-motivated when it comes to learning. First, I read the project documentation carefully and try to understand the existing codebase. Second, I am never afraid to ask smart questions when I get stuck after trying to solve it myself. Finally, I spend my spare time reading technical blogs and practicing new tools by building small projects.

3. Core Backend Stack (Golang, C#, Microservices, Docker)

Question 11: What are Goroutines in Go, and why are they useful?

Answer: Goroutines are very lightweight threads managed by the Go runtime. Unlike traditional threads, they use very little memory. We can easily run thousands of Goroutines at the same time. They are useful because they allow our application to handle many tasks concurrently.

Question 12: What is Dependency Injection (DI) in ASP.NET Core?

Answer: Dependency Injection is a design pattern that helps us write loose, flexible code. Instead of creating an object inside a class using the 'new' keyword, the framework automatically provides or 'injects' the object from the outside. This makes our code much easier to test, maintain, and Change.

Question 13: In a Microservices architecture, should all services share the same database?

Answer: No, they should not share the same database. In a microservice architecture, we use a pattern called 'Database per Service'. This means each microservice has its own separate database. This is important because if one database goes down, only that specific service stops Working.

Question 14: Why do we use Docker in backend development?

Answer: We use Docker because it packages our application code along with all its dependencies into a single container. This ensures that the application runs exactly the same way everywhere. It solves the famous problem: 'It works on my machine, but not in production.'

Certainly! Here are the same 4 essential practice questions and high-quality answers written in professional English, formatted exactly how you should present them during your AI interview:

1. JS/TS Proficiency (Coding Challenge)

Question: "Write a JavaScript function `reverseString(str)` that reverses a string without using any built-in reverse methods (like `.reverse()`)."

Ideal Code Answer (JavaScript):

```
function reverseString(str) {  
  let reversed = "";  
  // Loop through the string backwards starting from the last character  
  for (let i = str.length - 1; i >= 0; i--) {  
    reversed += str[i];  
  }  
  return reversed;  
}
```

// Test case

```
console.log(reverseString("zara")); // Output: "araz"
```

How to explain your logic to the AI:

"To solve this problem without using built-in array or string methods, I initialized an empty string called `reversed`. Then, I implemented a `for` loop that starts at the very last index of the input string and decrements down to zero. In each iteration, I append the current character to my `reversed` variable. This allows me to reverse the string efficiently with a time complexity of $O(n)$ and a space complexity of $O(n)$."

2. Fullstack Fundamentals (Conceptual Question)

Question: "Why would a development team choose to use GraphQL over a traditional REST API in a modern web application?"

Ideal Answer:

"The primary reason to choose GraphQL over REST is to solve the issues of **over-fetching** and **under-fetching** data. In a traditional REST architecture, endpoints return a fixed data structure. If a front-end component only needs a user's name, calling a REST endpoint might still return their entire profile, address, and history, which wastes network bandwidth.

Conversely, GraphQL allows the client to request exactly the specific data fields it needs, and nothing more. Furthermore, GraphQL utilizes a single endpoint, meaning clients can fetch multiple related resources in a single network request, significantly improving application performance on slower mobile networks."

3. Problem Solving & Debugging (Code Analysis)

Question: "What is the bug in the following JavaScript snippet? What will it output, and how do you fix it?"

```
for (var i = 1; i <= 3; i++) {  
  setTimeout(function() {  
    console.log(i);  
  }, 1000);  
}
```

Ideal Answer:

"The unexpected behavior in this code is that instead of printing 1, 2, and 3, it will print 4, 4, and 4 after a one-second delay.

The Bug: This happens because the `var` keyword is function-scoped, not block-scoped. The asynchronous `setTimeout` callbacks do not execute until the synchronous `for` loop has completely finished running. By the time the callbacks execute, the shared variable `i` has already been incremented to 4.

The Fix: To fix this bug, we should replace `var` with `let`. Because `let` is block-scoped, a completely new instance of `i` is bound to each individual iteration of the loop.

Corrected Code:

```
for (let i = 1; i <= 3; i++) {  
  setTimeout(function() {  
    console.log(i);  
  }, 1000);  
} // Output will now correctly be: 1, 2, 3  
..."
```

4. Ownership & Learning Velocity (Behavioral Question)

Question: "Can you describe how you take ownership when a critical bug or issue is discovered in your codebase?"

Ideal Answer:

"Whenever a major bug is identified in a project I am working on, my immediate response is to take absolute accountability rather than looking for excuses or placing blame. I approach the situation using three key steps:

- **First, Assessment and Isolation:** I immediately prioritize reproducing the bug in a local environment to isolate the root cause, utilizing debugging tools and checking system logs.
- **Second, Communication:** If the fix requires significant time or changes to production, I proactively communicate the issue to my team lead or stakeholders so we can mitigate any potential risks.
- **Third, Resolution and Prevention:** Once the fix is verified, I deploy it carefully. To prevent the issue from recurring, I write comprehensive unit tests covering that specific edge case and document the lessons learned for the team. I believe that

true ownership means managing a problem from the moment it is found until a permanent safeguard is built."

If you want to practice more, let me know if you would like to see English answers for **TypeScript interfaces vs types** or **SQL vs NoSQL databases**!

No problem at all! Let's make the answers **much simpler and easier to say** during your interview.

Here are the same 4 questions written in very simple, short sentences.

1. JS/TS Proficiency (Coding Challenge)

Question: "Write a JavaScript function to reverse a string without using `.reverse()`."

Simple Code:

```
function reverseString(str) {  
  let reversed = "";  
  // Loop backwards from the last letter  
  for (let i = str.length - 1; i >= 0; i--) {  
    reversed += str[i];  
  }  
  return reversed;  
}
```

What to say to the AI:

"First, I create an empty string called `reversed`. Then, I use a `for` loop. This loop starts from the last letter of the word and goes backward to the first letter. In every step, it adds the letter to my `reversed` string. This is a fast and simple way to flip a string."

2. Fullstack Fundamentals (Conceptual Question)

Question: "Why do we use GraphQL instead of REST API?"

What to say to the AI:

"We use GraphQL because it lets the frontend ask for **exactly** the data it needs."

With REST API, sometimes we get too much extra data that we do not need. This wastes internet speed. With GraphQL, we only get what we ask for. Also, GraphQL uses only one link, so the app loads faster on mobile phones."

3. Problem Solving & Debugging (Code Analysis)

Question: "What is the bug in this code and how do you fix it?"

```
for (var i = 1; i <= 3; i++) {  
  setTimeout(function() { console.log(i); }, 1000);  
}
```

What to say to the AI:

"The problem is that this code will print **4, 4, 4** instead of 1, 2, 3.

Why? Because `var` does not stay inside the loop. The loop finishes very fast and `i` becomes 4 before the timer prints it.

The Fix: We just change `var` to `let`. `let` creates a fresh copy of `i` for every step of the loop. Now, it will correctly print 1, 2, 3."

4. Ownership & Learning Velocity (Behavioral Question)

Question: "What do you do when you find a big bug in your code?"

What to say to the AI:

"When I find a big bug, I follow three simple steps:

- **Step 1:** I do not blame anyone. I take responsibility and look at the code to find why it failed.
 - **Step 2:** I tell my team leader about the problem immediately so they know I am fixing it.
 - **Step 3:** After I fix the bug, I test it carefully. I also write down notes so the same mistake does not happen again. I believe a good engineer always takes care of their mistakes."
-

These words are easy to remember and speak. Would you like me to make any other **programming topics** simple for you?

